Pandas and NumPy

Work with data efficiently • Perform numerical computations • Analyze and manipulate datasets

NumPy | pandas

From arrays to insights!



1. NumPy Basics

- Fast element wise array functions
- Multidimensional Array
- Data Processing using arrays
- File I/O with arrays

1.1 Fast Element wise Array Functions

Element wise operation

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
squared = np.square(arr)
print(squared)   # [ 1  4  9 16 25]
```

Output:

```
[ 1  4  9 16 25]
```

NumPy applies functions to each element of the array at once.

1.2 Multidimensional Array

2D array (rows, columns)

```
import numpy as np
arr = np.array([[1, 2, 3],
                [4, 5, 6]])
print(arr.shape)   # (2, 3)
print(arr)
```

Output:

```
(2, 3)
[[1 2 3]
 [4 5 6]]
```

1	2	3
4	5	6

rows

columns

1.3 Data Processing using Arrays

Statistical operations

```
import numpy as np
arr = np.array([10, 20, 30, 40])
mean = np.mean(arr)
std = np.std(arr)
print(mean, std)      # 25.0 11.18
```

Output:

```
25.0 11.180339887498949
```

Use vectorized operations for faster processing.

1.4 File I/O with Arrays

Save and load data

```
import numpy as np
# Saving array to a file
np.savetxt('data.csv', arr, delimiter=',')
# Loading array from a file
loaded_arr = np.loadtxt('data.csv', delimiter=',')
print(loaded_arr)
```

data.csv → save/load → NumPy Array

2. Pandas

- Data Structures
- Essential Functionality
- Summarizing and Computing Descriptive Statistics
- Handling Missing Data
- Hierarchical Indexing

pandas

Organize · analyze · explore data!



2.1 Data Structures

Series & DataFrame

```
import pandas as pd
# Series (1D)
s = pd.Series([10, 20, 30], index=['a','b','c'])
# DataFrame (2D)
df = pd.DataFrame({
    'Name': ['Alice','Bob','Charlie'],
    'Age': [25, 30, 35]
})
print(s)
print(df)
```

Series:

```
a    10
b    20
c    30
dtype: int64
```

DataFrame:

	Name	Age
0	Alice	25
1	Bob	30
2	Charlie	35

2.2 Essential Functionality

Load and explore data

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.head())    # first 5 rows
print(df.info())    # summary
```

Output (head):

	Name	Age	City
0	Alice	25	Delhi
1	Bob	30	Mumbai
2	Charlie	22	Bangalore

Output (info):

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3):
 #   Column  Non-Null Count  Dtype
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   City    3 non-null      object
dtypes: object(2), int64(1)
```

2.3 Summarizing and Computing Descriptive Statistics

Summary statistics

```
import pandas as pd
df = pd.DataFrame({'Age': [25, 30, 35, 40]})
print(df.describe())
```

Output:

	Age
count	4.000000
mean	32.500000
std	7.905694
min	25.000000
25%	28.750000
50%	32.500000
75%	36.250000
max	40.000000

Provides mean, std, min, max and more.

2.4 Handling Missing Data

Detect and handle missing values

```
import pandas as pd
df = pd.DataFrame({'Name': ['Alice','Bob','Charlie'],
                   'Score': [85, None, 92]})
print(df.isnull())    # Check missing values
df = df.fillna(0)     # Fill missing values
print(df)
```

Output:

	Name	Score
0	False	False
1	False	True
2	False	False

Use fillna(), dropna(), or interpolate().

2.5 Hierarchical Indexing

Multi-level index

```
import pandas as pd
arrays =[['A','A','B','B'],['1','2','1','2']]
index = pd.MultiIndex.from_arrays(arrays, names='Group','Number'))
df = pd.DataFrame({'Values': [10, 20, 30, 40]}, index=index)
print(df)
```

Output:

Group	Number	Values
A	1	10
A	2	20
B	1	30
B	2	40

Useful for complex data organization.

Arrays → Clean → Analyze → Visualize → Make Better Decisions

= Better data Better insights!

Pandas and NumPy: NumPy Basics

Work with Arrays, Perform Computations, and Analyze Data!

Pandas and NumPy are super powerful for data analysis!

Let's explore NumPy first, and then see how it helps with Pandas!

NumPy + pandas

1. What is NumPy?

NumPy is the foundation for numerical computing in Python. It provides:

- Fast multi-dimensional arrays
- Powerful mathematical functions
- Efficient data handling



2. Creating Arrays

Use np.array() to create arrays.

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr)
```

Output:
[1 2 3 4]

1	2	3	4

3. Important Operations

Perform common operations easily.

- Arithmetic: +, -, *, /
- Array functions: sum(), mean()
- Reshape: change shape
- Indexing & Slicing: arr[1:3]

0	1	2
3	4	5

```
arr = np.arange(6).reshape(2, 3)
```

4. Multidimensional Arrays

Work with 2D and higher-dimensional data.



Shape: (2, 2, 2)

5. NumPy Functions

Use useful functions for analysis.

- np.mean(), np.std()
- np.linspace()
- np.random.rand()
- np.dot(), np.transpose()



6. Pandas for Data Analysis

Pandas builds on NumPy and provides easy-to-use data structures.

- Series (1D data)
- DataFrame (tabular data)
- Read & explore data



7. Best Practices

- Use vectorized operations (instead of loops).
- Choose the right data type (e.g., int32, float32).
- Handle missing data.
- Work with large datasets efficiently.



8. Summary & Key Takeaways

- NumPy provides fast array operations.
- Pandas makes data analysis easier.
- Together, they power data science!

Start practicing with NumPy and Pandas today!



Learn NumPy, Explore Pandas, and unlock the power of data!

NumPy's element-wise functions let us perform operations on entire arrays at once — super fast!

Fast Element-wise Array Functions in NumPy

Write less code, Get results faster!

These functions are vectorized, so they work on whole arrays at once!

NumPy

1. What are Element-wise Functions?

They apply an operation to each element of an array, without using explicit loops.

1	× 2	2
2		4
3		6

Example:

```
arr = np.array([1, 2, 3])
arr * 2  # [2, 4, 6]
```

2. Common Fast Functions

- Arithmetic: add, subtract, multiply, divide
- Math: sqrt, sin, cos, exp, log
- Comparison: greater, less, equal, etc.
- Statistical: mean, sum, std, min, max

3. Examples in Action

Arithmetic Operations:

```
np.add(a, b), np.multiply(a, b)
```

Math Functions:

```
np.sqrt(arr), np.log(arr), np.sin(arr)
```

Comparison:

```
np.greater(arr, 0)
```

```
a = [1, 4, 9]
b = [2, 3, 4]
```

```
a + b = [3, 7, 13]
```

No loops! Just one line!

4. Useful Functions

- `np.abs()` – Absolute value
- `np.clip()` – Limit values
- `np.where()` – Conditional selection
- np.maximum(), `np.minimum()` – Element-wise max/min
- `np.isfinite()` – Check for finite values

5. Broadcasting Power

NumPy can automatically broadcast smaller arrays to match larger ones.

[1, 2, 3] + 5 → [6, 7, 8]

Example:

```
arr + 5  # Adds 5 to every element
```

6. Performance Tips

- Use vectorized functions instead of loops.
- Choose appropriate data types (e.g., float32).
- Leverage ufuncs for speed.
- Use in-place operations (e.g., +=).

Vectorization = Speed!

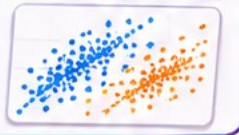
7. Advanced Functions

- np.sum(), np.mean(), np.std()
- np.exp(), np.log1p()
- np.square(), np.power()
- np.maximum.reduce()
- np.gradient()

8. Real-World Applications

- Data preprocessing (normalization, scaling)
- Image processing (pixel operations)
- Scientific computing (simulations)
- Machine Learning (feature scaling)





Master these functions to work with data more efficiently!

Fast. Simple. Powerful — That's NumPy!

Multidimensional Array in NumPy

Work with Arrays Beyond 1D!

A Multidimensional Array is an array with more than one dimension!

So, we can work with 2D, 3D and even higher dimensions!

1. What is a Multidimensional Array?

An array with 2 or more dimensions, like an table (2D) or a cube (3D).



Example (3D Array):

```
arr = np.array([[[1, 2],
                 [3, 4]],
                [[5, 6],
                 [7, 8]]])
```

2. Creating Multidimensional Arrays

Use np.array() with nested lists.

```
arr_2d = np.array([[1, 2, 3],
                   [4, 5, 6]])
arr_3d = np.array([[[1,2],[3,4]],
                   [[5,6],[7,8]]])
```

Common Methods:

- np.zeros((m, n))
- np.ones((m, n))
- np.full((m, n), value)



3. Understanding Dimensions

Use .ndim, .shape and .size

```
arr = np.array([[1,2,3],[4,5,6]])
print(arr.ndim)    # 2
print(arr.shape)   # (2, 3)
print(arr.size)    # 6
```

ndim	shape	size
2	(2, 3)	6

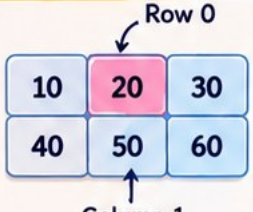
Shape shows the structure of the array!

4. Accessing Elements

Use indexing and slicing.

```
arr = np.array([[10,20,30],
                [40,50,60]])

print(arr[0, 1])   # 20
print(arr[:, 1])   # [20 50]
```



5. Common Operations

Perform mathematical operations element-wise.

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])
print(a + b)   # Add
print(a * b)   # Multiply
```

```
[ 6   8 ]
[10  12]
```

Operations are applied to each element!

6. Useful Functions

Explore powerful functions:

- np.sum() – Sum of all elements
- np.mean() – Mean value
- np.reshape() – Change shape
- np.transpose() – Swap axes
- np.concatentate() – Join arrays

7. Advanced Concepts

- **Broadcasting** – Apply operations with different shapes.
- **Axis** – Specify axis for operations (e.g. sum(axis=0)).
- **Reshape** – Change array shape.
- **Flatten** – Convert to 1D array.
- **Generate grids** – np.meshgrid().



8. Real-World Applications

- Data Analysis (with Pandas)
- Image Processing
- Scientific Computing
- Machine Learning

Key Takeaways

- Multidimensional arrays are essential for working with complex data.
- Use **.shape, .ndim** and slicing to manage them.
- Leverage NumPy's fast functions for efficient computation.
- Practice with real-world data!

Explore, Experiment, and Unlock the Power of NumPy!

Multidimensional Arrays = More Power for Data Science!

Data Processing Using Arrays

Clean • Transform • Analyze • Visualize

Arrays are not just for storing data... They help us process data too!

Let's explore how to process data efficiently with NumPy!

1. What is Data Processing?

Data processing means cleaning, transforming and analyzing data to get useful insights.



Examples:

- Filtering data
- Calculating statistics
- Preparing data for ML

2. Working with Arrays

Arrays make data processing fast!

```
import numpy as np

data = np.array([10, 20, 30, 40])
print(data * 2)   # [20 40 60 80]
```

Arrays support vectorized operations for element-wise processing.

3. Common Processing Tasks

Filtering
`data[data > 0]`
→ Positive values

Transformation
`np.sqrt(data)`
→ Square root

Aggregation
`np.mean(data)`
→ Average

Sorting
`np.sort(data)`
→ Sorted data

4. Useful Functions



- `np.sum()` – Sum of elements
- `np.mean()` – Mean value
- `np.std()` – Standard deviation
- `np.min()` / `np.max()` – Min/Max
- `np.unique()` – Unique values
- `np.cumsum()` – Cumulative sum

These functions help in quick data analysis!

5. Data Cleaning



Real-world data often has missing or invalid values. Handle them!

- `np.isnan()` – Find missing data
- `np.nan_to_num()` – Replace NaN
- `np.clip()` – Limit values

Clean data = better results!

6. Transforming Data

Reshape, normalize and scale data.

```
data = data.reshape(2, 2)

normalized = (data - data.min()) /
(data.max() - data.min())
```

Useful for machine learning and data visualization.



7. Real-World Applications

- Data Analysis & Business Intelligence
- Image Processing
- Scientific Research
- Machine Learning
- Finance & Healthcare

Arrays power data-driven decisions!

8. Best Practices

- Use vectorized operations
- Choose appropriate data types (e.g., int32, float32)
- Handle missing values
- Avoid unnecessary copies
- Profile for performance

Key Takeaways

- Arrays help process data efficiently.
- Use NumPy functions for cleaning, transforming and analyzing data.
- Good data processing leads to better insights!



File I/O with Arrays in NumPy

Read • Write • Save • Analyze

Let's learn how to work with files using NumPy arrays!

So we can load, save and process data easily?

1. Why File I/O?

File Input/Output lets us read data from files and save our results for future use.

Common Formats:

- CSV
- TXT
- Excel (via Pandas)

2. Reading Files

Use np.loadtxt() or np.genfromtxt() to load data.

```
import numpy as np
data = np.loadtxt('data.csv',
        delimiter=',')
print(data)
```

data.csv

3. Writing Files

Use np.savetxt() to save arrays.

```
np.savetxt('output.csv',
data, delimiter=',',
fmt='%0.2f')
```

Save your results easily!

output.csv

4. Working with Different Formats

You can also work with binary files for efficient storage.

```
np.save('array.npy', data)
loaded = np.load('array.npy')
```

.npy files are faster!

5. File Operations

Important modes:

- 'r' – Read
- 'w' – Write (overwrite)
- 'a' – Append
- 'rb' – Read Binary
- 'wb' – Write Binary

6. Handling Large Data

Use efficient methods for big files:

- Load data in chunks
- Use memory mapping (np.memmap)
- Leverage Pandas for complex data

This helps when data is huge!

7. Real-World Applications

- Scientific Research (experiments)
- Finance (market data)
- Image Processing
- Machine Learning (dataset handling)
- Data Science Projects

8. Key Takeaways

- Use np.loadtxt() and np.savetxt() for text data.
- Use np.save() and np.load() for NumPy arrays.
- Choose the right file mode.
- Handle large data efficiently.
- Combine with Pandas for advanced file handling.

File I/O with Arrays makes your data portable, reusable and easy to analyze!

Work with files. Unlock the power of data!

Pandas – Data Structures

Series & DataFrame

Organize • Analyze • Explore

pandas

Pandas helps us work with structured data easily!

Let's learn the core data structures in Pandas!

1. What is Pandas?

Pandas is a powerful Python library for data analysis and manipulation.

- Easy data handling
- Fast & Flexible
- Built on NumPy

pandas

2. Series

A 1D labeled array of data.

```
import pandas as pd
s = pd.Series([10, 20, 30],
    index=['a', 'b', 'c'])
print(s)
```

a	b	c
10	20	30

- Labels (Index)
- Supports any data type

3. DataFrame

A 2D labeled data structure (like a table).

```
df = pd.DataFrame({
   'Name': ['Alice','Bob'],
   'Age': [25, 30] })
print(df)
```

	Name	Age
0	Alice	25
1	Bob	30

- Rows & Columns
- Heterogeneous data



4. Creating Data Structures

From lists, dicts or arrays.

```
pd.Series([1,2,3])
pd.DataFrame([[1,2],[3,4]],
             columns=['A','B'])
```



Series



DataFrame

5. Exploring Data

Get to know your data.

```
df.head()     # first rows
df.info()     # summary
df.describe() # statistics
```

- Check shape
- View data types
- Get summary stats



6. Key Features

- Indexing & Selection
- Filtering
- Grouping & Aggregation
- Handling Missing Data
- Data Merging & Joining
- Sorting & Ranking



7. Working with Data

Manipulate data with ease.

```
df['Age'] > 25
df.groupby('Dept').mean()
df.dropna()
```

- Transform data
- Perform calculations
- Combine datasets



8. Useful Functions

Powerful methods for analysis.

```
df.mean()  df.sum()
df.sort_values()
df.pivot_table()
```

- Statistical analysis
- Data reshaping
- Time series tools



9. Real-World Applications

- Finance – stock analysis
- Healthcare – patient data
- Marketing – customer insights
- Science – research data
- Machine Learning – training data





10. Key Takeaways

- Series is 1D, DataFrame is 2D
- Pandas makes data analysis easy
- Explore, clean, transform & visualize
- A must-know library for Data Science!

Pandas = Your Data Analysis Superpower!



Essential Functionality in NumPy & Pandas

Manipulate • Analyze • Discover

NumPy gives us powerful array operations, and Pandas makes data analysis easy!

From basic operations to advanced analysis — these libraries have everything!

Explore the essentials!

1. Array Operations (NumPy)

Perform fast element-wise operations.

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a + b) # [5 7 9]
```



- Arithmetic: +, -, *, /
- Universal functions (ufuncs)
- Matrix operations

2. Data Structures (Pandas)

Work with labeled data.

```
import pandas as pd
df = pd.DataFrame({
    'Name': ['Alice','Bob'],
    'Age': [25, 30]
})
```

Name	Age
Alice	25
Bob	30

- Series & DataFrame
- Indexing & slicing
- Handling missing data

3. Data Selection & Filtering

Get the data you need.

```
df[df['Age'] > 25]
df.loc[:, 'Name']
df.iloc[0]
```



- Boolean indexing
- loc and iloc
- Query method

4. Data Manipulation

Transform and reshape your data.

```
df['Age_plus'] = df['Age'] + 1
df.groupby('Age').mean()
df.melt()
```



- Apply, map, replace
- Groupby & aggregate
- Reshape & pivot

5. Statistical Analysis

Find insights from your data.

```
df.describe()
df.corr()
df.quantile(0.75)
```



- Summary stats
- Correlation
- Advanced analytics

6. Handling Dates & Time

Work with time series data.

```
pd.to_datetime(df['Date'])
df.set_index('Date')
df.resample('M').mean()
```



- DatetimeIndex
- Resampling
- Time-based operations

7. Input/Output

Read and write data from different sources.

```
df.to_csv('data.csv')
pd.read_excel('file.xlsx')
pd.read_json('data.json')
```



- CSV, Excel, JSON
- Parquet & HDF5
- Efficient storage

8. Essential Features

Powerful tools for data analysis.

- Merge & Join
- Concatenate
- Handle Duplicates
- Missing Data (fillna, dropna)
- Apply Custom Functions
- Plotting (with Matplotlib)



Key Takeaways

- NumPy for efficient numerical operations.
- Pandas for flexible data manipulation.
- Use essential functions to clean, analyze and visualize data.
- These tools are must-haves for every data scientist!



Master the Essentials – Unlock the Power of NumPy & Pandas!

Summarizing and Computing Descriptive Statistics in NumPy & Pandas

Understand • Analyze • Discover

Descriptive statistics help us summarize data with key numbers!

Let's compute mean, median, and more!

Turn Data into Insights!

1. What are Descriptive Statistics?

They are summary measures that describe the main features of data.

- Central Tendency
- Dispersion
- Distribution Shape



2. Key Measures

Mean	Average value
Median	Middle value
Mode	Most frequent value
Variance	Spread of data
Std Dev	Standard deviation
Min / Max	Smallest & largest

3. Using NumPy

Quickly compute statistics with NumPy functions.

```
import numpy as np
a = np.array([1, 2, 3, 4, 5])
print(np.mean(a))   # 3.0
print(np.std(a))    # 1.41
```

4. Using Pandas

Get detailed summaries with Pandas.

```
import pandas as pd
df = pd.DataFrame({
    'A':[10, 20, 30, 40]
})
print(df.describe())
```

5. Essential Functions

- `df.describe()` – Summary stats
- `df.mean()`, `df.median()`
- `df.std()`, `df.var()`
- `df.min()`, `df.max()`
- `df.quantile()` – Percentiles
- `df.info()` – Data overview

describe() Output

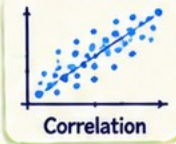
count	4.0
mean	25.0
std	12.91
min	10.0
25%	17.5
50%	25.0
75%	32.5
max	40.0

6. Advanced Statistics

- Skewness – Asymmetry
- Kurtosis – Tail heaviness
- Correlation – Relationship
- Covariance – Joint variation
- Percentiles – 25th, 50th, 75th

```
df['A'].skew()
df['A'].kurt()
```





7. Grouped Statistics

Summarize data by groups.

```
df.groupby('City').agg({
    'Sales': ['mean', 'sum']
})
```

City	mean	sum
NY	120.0	360
LA	90.0	270

8. Handling Missing Data

Get accurate results even with missing values.

- `df.dropna()`
- `df.fillna()`
- `df.mean(skipna=True)`



9. Real-World Applications

- Business Analytics
- Healthcare
- Finance
- Science & Research
- Machine Learning

10. Key Takeaways

- Use **describe()** for a quick overview.
- Choose the right measure (mean, median, etc.).
- Explore data distribution (skewness, kurtosis).
- Group and summarize for deeper insights!

Better Statistics = Smarter Decisions!

Summarizing Data with NumPy & Pandas – Uncover the Big Picture!

Handling Missing Data in NumPy & Pandas

Clean Data • Better Insights!

Missing data can affect your analysis. Let's learn how to handle it!

Clean data leads to more accurate results!

Don't ignore NaN!

1. What is Missing Data?

Missing data means some values are absent or unknown in your dataset.

Examples: NaN, None, blank values, or ?

Age	Income	City
25	50000	Delhi
NaN	42000	Mumbai
30	NaN	Chennai

2. Common Causes

- Incomplete data collection
- Sensor or device errors
- Data entry mistakes
- Data not available

3. Detecting Missing Data

```
df.isnull()          # Check for NaN
df.isna()            # Same as isnull()
df.isnull().sum()    # Count missing
```

Name	Age	Salary
Alice	25	50000
Bob	NaN	60000

4. Handling Techniques

Drop Missing Data

```
df.dropna()
# Remove rows or columns
```

Fill with a Value

```
df.fillna(0)
# Replace NaN with 0
```

Fill with Statistics

```
df.fillna(df.mean())
# Use mean/median
```

Interpolate

```
df.interpolate()
# Estimate values
```

5. Useful Options

- axis : 0 (rows) or 1 (columns)
- inplace : Update original data
- limit : Fill only limited gaps
- method : forward/backward fill

```
df.fillna(method='ffill', limit=1)
```

6. Advanced Techniques

- Drop rows/columns with too many missing values
- Use Scikit-learn for imputation
- Handle missing data by groups

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='mean')
df[['Age','Income']] = imputer.fit_transform(
  df[['Age','Income']])
```

7. Impact on Analysis

- Biased results
- Loss of important data
- Reduced model accuracy

Handle missing data wisely!

8. Real-World Applications

- Healthcare (research data)
- Finance (missing transactions)
- Business Analytics (surveys)
- Machine Learning (incomplete features)

Key Takeaways

- Identify missing data using isnull() or isna().
- Choose the right strategy: drop, fill or impute.
- Use appropriate methods (mean, median, etc.).
- Validate your data after handling missing values.

Clean Data = Reliable Insights!

Hierarchical Indexing in Pandas

Work with Multi-Level Indexes for Powerful Data Analysis!

Hierarchical Indexing helps us work with multi-level data!

It's perfect for complex datasets!

1. What is Hierarchical Indexing?



It is a multi-level index (MultiIndex) that lets you index data using multiple keys (levels).

Think of it as a nested index!

2. Example: Multi-Level Index

	Region			
	East	East	West	West
	A	B	A	B
Sales	100	120	90	110

Two levels!

3. Creating a MultiIndex

```
import pandas as pd
index = pd.MultiIndex.from_product(
    [['East', 'West'], ['A', 'B']])
df = pd.DataFrame({'Sales':[100,120,90,110]},
                  index=index)
```

Use from_product(), from_tuples() or from_arrays()

4. Accessing Data



- Use .loc[] with multiple levels:
  `df.loc[('East', 'A')]`
- Slice by one level:
  `df.loc['East']`
- Cross-section:
  `df.xs('A', level='Product')`

5. Useful Features

- Swap levels: `df.swaplevel()`
- Sort levels: `df.sort_index()`
- Reset index: `df.reset_index()`
- Drop level: `df.droplevel()`



Makes complex data easy to explore!

6. Real-World Use Cases



Sales Analysis



Healthcare Data



Web Analytics



Finance Data

Hierarchical indexing is widely used in real-world datasets!

7. Advantages

- Organizes complex data clearly
- Supports multi-level analysis
- Improves data selection and performance

8. Key Takeaways

- Hierarchical Indexing = Multi-Level Index
- Use MultiIndex for complex datasets
- Access data with `.loc[]`, `.xs()`
- Enhance analysis with `swaplevel()`, `sort_index()` and more!

Organize, Analyze, Discover!

Explore Multi-Level Data!

Master Hierarchical Indexing for Deeper Insights!